

2. Методи и функции (подпрограми) в C++. Функции от по-висок ред. Функции предикати. Рекурсивно програмиране и рекурсивни структури от данни. Примерен код на C++.

1. Функции (подпрограми)

Функцията е именуван блок от код, който изпълнява конкретна задача и може да бъде извикан многократно. В C++ функциите са основният механизъм за декомпозиция - разбиване на голям проблем на малки части.

Предимства: Преизползването на код, по-лесно тестване, по-добра четимост, скриване на детайли

1.1 Анатомия на функция

```
// Прототип
int sum(int a, int b);

// Дефиниция
int sum(int a, int b) { // тип резултат име (параметри)
    return a + b; // тяло
}
```

```
int main() {
    int r = sum(3, 4);
    return 0;
}
```

- Прототип - казва на компилатора за съществуването на функцията
- Формални параметри - Локални променливи, получаващи стойности при извикване
- Фактически параметри - Конкретни стойности/изрази при извикване
- `return` - прекратява функцията и връща стойност

2. Предаване на параметри

2.1 По стойност

```
void udvoi(int x) {
    x = x * 2; // промяната е само в копието
}
```

2.2 По референция

Функцията работи директно върху оригинала чрез псевдоним (`&`)

```
void udvoi(int& x) {
    x = x * 2; // променя оригинала
}
```

2.3 По указател

Подава се адресът на променливата

```
void udvoi(int* x) {
    *x = (*x) * 2;
}
```

```
}
```

2.4 Параметри по подразбиране

```
double power(double base, int exp = 2) {  
    double res = 1;  
    for (int i = 0 ; i < exp ; i++) {  
        res *= base;  
    }  
  
    return res;  
}  
  
// power(3) => 9  
// power(3, 3) => 27
```

Правило: параметрите с подразбиращи стойности винаги са последни в списъка.

3. Претоварване на функции (Function Overloading)

C++ позволява няколко функции с едно и също име, но различни параметри. Компиляторът избира правилната версия по броя/типовете аргументи.

```
int max(int a, int b) { return a > b ? a : b; }  
double max(double a, double b) { return a > b ? a : b; }  
int max(int a, int b, int c) { return max(max(a, b), c); }
```

Типът на резултата не участва в разграничаването, а само параметрите.

4. Шаблонни функции

Когато логиката е еднаква за различните типове, използваме `template`:

```
template <typename T>  
T maxT(T a, T b) {
```

```

    return a > b ? a : b;
}

// maxT<int>(3, 7)      => 7
// maxT<double>(3.1, 2.9) => 3.1
// maxT<string>("abc", "xyz") => "xyz"

```

5. Функции от по-висок ред

Функция от по-висок ред е функция, която приема друга функция като параметър или връща функция като резултат.

5.1 Указатели към функции

```

// Тип: указател към функция, приемаща int и връщаща int
typedef int (*IntTransform)(int);

int double_(int x) { return x * 2; }
int int_square(int x) { return x * x; }

void applyToAll(int arr[], IntTransform transform) {}

int a[] = {1, 2, 3, 4};
applyToAll(a, 4, double_);
applyToAll(a, 4, square);

```

5.2 Lambda изрази

Lambda е анонимна функция, дефинирана на място.

```

auto add10 = [](int x) { return x + 10; };

```

5.3 Практически пример

```
std::vector<int> v = {1, 2, 3, 4, 5};
// transform (аналог на map) – удвоява всеки елемент
std::transform(v.begin(), v.end(), v.begin(), [](int x) { return x * 2 ;});
```

6. Функции предикати

Предикат е функция, която връща `bool` - тя проверява условие за даден елемент.

```
bool isEven(int n) { return n % 2 == 0; }
bool isPositive(int n) { return n > 0; }
```

6.1 Предикати с STL алгоритми

```
auto it = std::find_if(v.begin(), v.end(), isEven);

int br = std::count(v.begin(), v.end(), isEven);
```

6.2 Двоичен предикат

Предикат с 2 аргумента - използва се при сортиране:

```
bool poAbs(int a, int b) {
    return std::abs(a) < std::abs(b);
}

std::vector<int> v = {-5, 2, -1, 4};
std::sort(v.begin(), v.end(), poAbs);
```

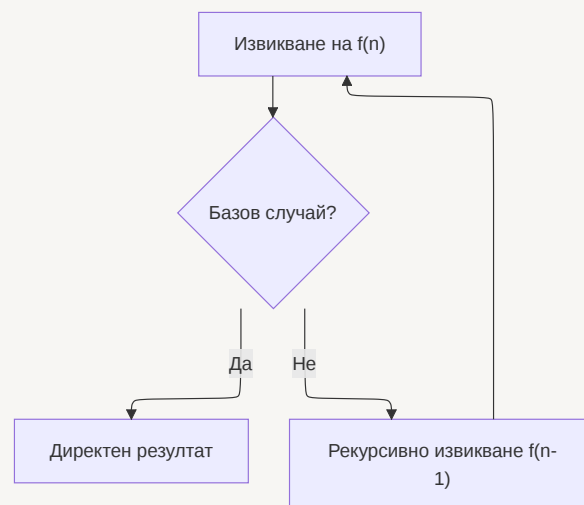
7. Рекурсивно програмиране

Рекурсия - функция, която извиква сама себе си с по-просто вариант на задачата.

7.1 Структура на всяка рекурсивна функция

flowchart TD

```
A["Извикване на f(n)"] --> B{"Базов случай?"}
B -- "Да" --> C["Директен резултат"]
B -- "Не" --> D["Рекурсивно извикване f(n-1)"]
D --> A
```



Задължителни елементи:

1. Базов случай - условие, при което рекурсията спира
2. Рекурсивно стъпало - задачата се свежда до по-прост вариант, клонящ към базовия случай

7.2 Примери

Факториел:

```
long long factorial (int n) {
    if (n <= 1) return 1; // базов случай
```

```
    return n * factorial(n - 1); // рек. стъпало
}
```

Числата на Фибоначи:

```
int fib(int n) {
    if (n <= 1) return n; // базов случай
    return fib(n - 1) + fib(n - 2);
}
```

7.3 Видове рекурсия

- Линейна - Едно рекурсивно извикване
- Разклонена - Повече от едно извикване
- Опашкова - Рекурсивното действие е последното действие
- Взаимна - `f()` вика `g()`, `g()` вика `f()`

// Опашкова рекурсия

```
int sumTail(int n, int acc = 0) {
    if (n == 0) return acc;
    return sumTail(n - 1, acc + n);
}
```

7.4 Стек на изискванията (Call Stack)

При всяко извикване на функция процесорът запазва в стека:

- Адрес на връщане
- Локалните промени и параметри

```
factorial(4)
└ factorial(3)
```

```
└ factorial(2)
  └ factorial(1) ← базов случай, започва размотаване
```

8. Рекурсивни структури от данни

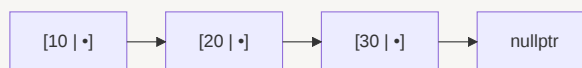
Рекурсивна структура - структура, дефинирана чрез самата себе си (съдържа указател към елемент от същия тип).

8.1 Свързан списък (Linked List)

```
struct Node {
    int data;
    Node* next; // указател към следващ Node – рекурсивна деф
иниция
}
```

graph LR

```
A["[10 | •]"] --> B["[20 | •]"]
B --> C["[30 | •]"]
C --> D["nullptr"]
```



```
void print(Node* head) {
    if(!head) return;
    cout << head->data << " ";
    print(head->next);
}

Node* find(Node* head, int val) {
    if (!head) return nullptr;
    if (head->data == val) return head;
}
```

```

    return find(head->next, val);
}

int length(Node* head) {
    if(!head) return 0;
    return 1 + length(head->next);
}

```

8.2 Бинарно дърво

```

struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;
}

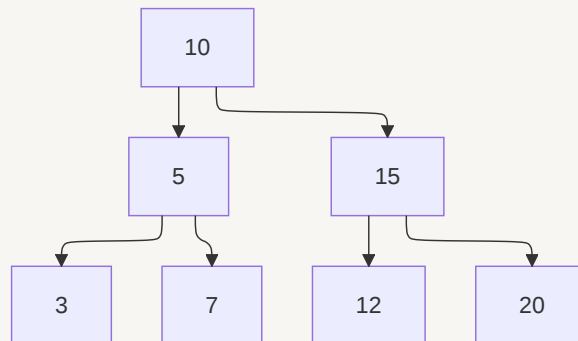
```

graph TD

```

A["10"] --> B["5"]
A --> C["15"]
B --> D["3"]
B --> E["7"]
C --> F["12"]
C --> G["20"]

```



Три вида обход:

```
// Inorder: ляво → корен → дясно (дава сортирана наредба при BST)
```

```
void inorder(TreeNode* root) {  
    if (!root) return;  
    inorder(root->left);  
    cout << root->data << " ";  
    inorder(root->right);  
}
```

```
// => 3 5 7 10 12 15 20
```

```
// Preorder: корен → ляво → дясно
```

```
void preorder(TreeNode* root) {  
    if (!root) return;  
    cout << root->data << " ";  
    preorder(root->left);  
    preorder(root->right);  
}
```

```
// => 10 5 3 7 15 12 20
```

```
// Postorder: ляво → дясно → корен
```

```
void postorder(TreeNode* root) {  
    if (!root) return;  
    postorder(root->left);  
    postorder(root->right);  
    cout << root->data << " ";  
}
```

```
// => 3 7 5 12 20 15 10
```

Вмъкване в BST:

```
TreeNode* insert(TreeNode* root, int val) {  
    if (!root) return new TreeNode{val, nullptr, nullptr};  
    if (val < root->data)
```

```
    root->left = insert(root->left, val);  
else  
    root->right = insert(root->right, val);  
return root;  
}
```